

APPENDIX

ML-25/26-05: Python Code Tool Interpreter for Chart Creation
Author: Md Sohel Rana (1428128), Md Ashiqur Rahman (1393169)

Project requirement analysis.

The project was quite unique because modern Large Language Models (LLMs) are already capable of generating code independently. Therefore, an important question arose during the project development process that is why is there a need for this application, and how can it provide additional value beyond a standard LLM-based code generation system?

To better understand the project requirements and define the overall direction of the application, we utilized the LLM model OpenAI's ChatGPT as a supporting tool during the planning and design phases. We used it to explore how such an application should be developed, how the system architecture and workflow should be designed, and how the proposed solution could differentiate itself from conventional LLM-based coding assistants.

The discussions and prompts helped us analyse various aspects of the system, including application structure, architectural design patterns, feature implementation strategies, scalability considerations, and possible approaches for improving usability and efficiency. In particular, the LLM assisted us in identifying ways to make the application more unique and practical compared to a normal code-generating LLM model by focusing on workflow optimization, better integration mechanisms, and enhanced user interaction.

To gain a clearer understanding of the project scope and technical direction, we used the following prompt as an initial reference during the requirement analysis and design process.

“As part of our Software Engineering project, we are planning to develop an AI agent that can generate Python code based on user instructions. We have received some guidelines and requirements from our professor, which are attached here.

Large Language Models (LLMs) have demonstrated significant potential in generating executable code from natural language instructions, particularly in the domains of data analysis and visualization. Despite these capabilities, LLM-based agents typically lack the ability to execute the generated code autonomously. This project investigates the design and implementation of an intelligent agent based on the Microsoft Agent Framework using .NET and C#, augmented by a custom Code Interpreter Tool implemented as an agent plug-in. The agent translates user prompts—such as requests for charts or diagrams—into Python code for data visualization, while the Code Interpreter Tool is responsible for executing this code and generating the corresponding visual output. A central focus of the project is the robust mapping of user-provided data to executable code, as well as the interpretation and execution of dynamically generated Python scripts using appropriate libraries. Furthermore, the project explores strategies for efficiently handling large datasets and supporting the generation of multiple visualizations from complex user inputs. Key challenges addressed include scalability, data transfer between agent components, and the reliable execution of model-generated code. The outcome of this work aims to identify best practices for integrating code execution

capabilities into LLM-driven agent architectures for advanced data visualization tasks.

Now please tell me how this project could be developed. What should the plan be, and how can it be done”

AI Response:

Develop the project as a **.NET/C# AI visualization agent with a custom Python execution plug-in**. The agent should not only generate Python code; it should also validate it, execute it safely, and return generated chart files.

I found related course/project material in your file library about tool routing and argument mapping. It emphasizes a modular pipeline, tool-call tracking, argument validation, decoy/confusion testing, and measurable evaluation. That structure fits this project very well. Microsoft Agent Framework is also suitable because it supports agents that process inputs, call tools, and generate responses, and it provides workflows for controlled multi-step orchestration. ([Microsoft Learn](#))

1. Target system architecture

A good architecture would be:

User Prompt + Dataset



.NET / C# Application



Microsoft Agent Framework Agent



Prompt Builder + Dataset Profiler



LLM generates visualization plan + Python code



Code Validator



Custom Code Interpreter Tool



Python Sandbox executes code



Generated chart files + execution logs



Final answer to user

The important design decision is that the **LLM should not directly control the whole system**. It should only generate a structured visualization plan and Python code. The .NET application should validate inputs, manage files, run the code in a controlled environment, and collect outputs.

Microsoft's own Code Interpreter documentation describes the same general idea: an agent writes and executes code in a sandbox, which is useful for data analysis, mathematical computation, and file processing. ([Microsoft Learn](#)) For your project, however, the professor's requirement suggests building a **custom Code Interpreter Tool** instead of relying only on a hosted one.

2. Recommended project scope

For a strong but realistic Software Engineering project, define the first version like this:

Input supported:

CSV files

Excel files

JSON files

manual tabular data pasted by the user

Visualization supported:

bar chart

line chart

scatter plot

pie chart

histogram

box plot

heatmap

multiple charts from one prompt

Python libraries:

pandas

numpy

matplotlib

seaborn or plotly, optional

Output:

generated Python code

PNG/SVG chart files

execution status

error message if execution fails

final natural-language explanation

3. Main components to build

A. Agent application in .NET/C#

This is the main application. It receives the user request, loads the dataset, calls the LLM through Microsoft Agent Framework, invokes the custom tool, and returns the result.

Microsoft Agent Framework supports multiple model providers through chat clients, including Azure OpenAI, OpenAI, Foundry, Anthropic, Ollama, and custom providers, so you can keep the system provider-independent. ([Microsoft Learn](#))

Core classes could be:

AgentApp

- |— Program.cs
- |— AgentFactory.cs
- |— PromptBuilder.cs
- |— DatasetProfiler.cs
- |— VisualizationPlanner.cs
- |— CodeValidator.cs
- |— Tools/
 - | |— CodeInterpreterTool.cs
- |— Execution/
 - | |— PythonSandboxRunner.cs
 - | |— ExecutionResult.cs
- |— Output/
 - | |— ArtifactManager.cs

B. Dataset profiler

Before asking the LLM to write Python code, your system should inspect the uploaded dataset.

The profiler should extract:

column names

data types

number of rows

number of columns

missing values

sample rows

numeric columns

categorical columns

date/time columns

This is important because the LLM should not guess column names. It should receive a compact schema such as:

```
{  
  "file": "sales.csv",  
  "rows": 25000,  
  "columns": [  
    {"name": "Date", "type": "datetime"},  
    {"name": "Region", "type": "string"},  
    {"name": "Sales", "type": "float"},  
    {"name": "Profit", "type": "float"}  
  ],  
  "sampleRows": [  
    {"Date": "2025-01-01", "Region": "Europe", "Sales": 1200, "Profit": 300}  
  ]  
}
```

The agent then receives the user request plus this schema, not the whole dataset.

C. Visualization planner

Do not ask the LLM to generate Python immediately. First ask it to generate a structured plan.

Example output:

```
{  
  "charts": [  
    {
```

```
"chartType": "bar",  
"title": "Sales by Region",  
"x": "Region",  
"y": "Sales",  
"aggregation": "sum",  
"outputFile": "sales_by_region.png"  
}  
]  
}
```

This makes the system easier to validate. If the user asks:

“Create a bar chart showing total sales by region and a line chart showing monthly profit trend.”

The planner should return two chart specifications. Then the code generator writes Python based on those specifications.

D. Code generator

The agent should generate Python code with strict rules:

Read the dataset from the provided file path.

Use only allowed libraries.

Do not access the internet.

Do not delete files.

Save all charts into the output directory.

Return a JSON summary of generated files.

Use only real column names from the dataset profile.

The prompt should tell the model exactly where to save output:

Save every visualization to /outputs.

Use matplotlib.

Do not call plt.show().

Use plt.savefig().

Return generated file paths.

E. Code validator

Before execution, validate the Python code.

Minimum checks:

Does the code reference only existing dataset columns?

Does it save at least one output file?

Does it avoid dangerous imports such as os, socket, subprocess?

Does it avoid network access?

Does it avoid deleting or modifying system files?

Is the script syntactically valid?

For a university project, this validation layer is very important because it shows software engineering discipline, not just LLM prompting.

F. Custom Code Interpreter Tool

This is the most important part of the project.

The tool should expose one main function to the agent:

`ExecutePythonCode(string code, string datasetPath, string outputDirectory)`

It should return:

```
{  
  "success": true,  
  "stdout": "...",  
  "stderr": "",  
  "generatedFiles": [  
    "outputs/sales_by_region.png"  
  ],  
  "executionTimeMs": 1830  
}
```

For the first version, you can run Python using `ProcessStartInfo` from C#. For a stronger version, run the code inside Docker.

Recommended execution levels:

Basic version:

C# starts `python.exe` with a timeout.

Better version:

C# creates a temporary working directory and runs Python there.

Best version:

C# runs Python inside a Docker container with CPU, memory, timeout, and network restrictions.

Microsoft's hosted Foundry Code Interpreter uses an isolated sandbox and can generate files such as charts, which is the same safety principle your custom version should follow. ([Microsoft Learn](#))

4. Handling large datasets

Do **not** send large datasets to the LLM.

Use this approach instead:

Small dataset:

Send schema + sample rows.

Medium dataset:

Send schema + statistics + 5–20 sample rows.

Large dataset:

Send schema + column statistics + pre-aggregated summaries.

For large files, use one of these strategies:

Use pandas chunking.

Use DuckDB for SQL-like aggregation.

Generate intermediate summary tables.

Ask the LLM to write code that reads from the file path, not from prompt text.

Example:

User asks: "Show monthly sales trend for 2 million rows."

System should:

1. Detect date and sales columns.
2. Generate Python that groups by month.
3. Plot only the aggregated result.

This directly addresses the project's scalability requirement.

5. Supporting multiple visualizations

Use a two-step process:

Step 1: LLM creates visualization plan.

Step 2: LLM creates Python code for all charts.

Example plan:

```
{
  "charts": [
    {
      "chartType": "bar",
      "x": "Region",
      "y": "Sales",
      "aggregation": "sum",
      "outputFile": "chart_1_sales_by_region.png"
    },
    {
      "chartType": "line",
      "x": "Month",
      "y": "Profit",
      "aggregation": "sum",
      "outputFile": "chart_2_profit_by_month.png"
    }
  ]
}
```

Then the Python code should generate both images in one execution.

6. Development plan

Phase 1: Requirements and design

Define:

supported input formats

supported chart types
allowed Python libraries
security rules
expected output format
evaluation metrics
Deliverables:
requirements document
architecture diagram
use-case diagram
sequence diagram
risk analysis

Phase 2: Build the basic agent

Create a .NET console app or ASP.NET Web API.

Implement:

AgentFactory

PromptBuilder

basic chat with Microsoft Agent Framework

one simple tool registration

At this stage, test with prompts like:

“Generate a bar chart of sales by region.”

The output can just be Python code, without execution yet.

Phase 3: Build the Code Interpreter Tool

Implement:

temporary folder creation

Python script writing

process execution

timeout handling

stdout/stderr capture

generated file detection

error reporting

For example:

/input/data.csv

/work/script.py

/outputs/chart.png

The tool should never execute directly in the project root.

Phase 4: Add dataset profiling and column mapping

Implement:

CSV/Excel parser

column type detection

sample row extraction

natural-language-to-column matching

missing column detection

This is where your project becomes stronger than a simple “LLM writes Python” demo.

Example:

User says: “Plot revenue by country.”

Dataset has columns:

CountryName, TotalRevenue, Date

System maps:

revenue → TotalRevenue

country → CountryName

If mapping is uncertain, the agent should ask a clarification question.

Phase 5: Add code validation and repair

Execution will fail often in early versions. Build an automatic repair loop:

1. Generate code.
2. Validate code.
3. Execute code.

4. If error occurs, send error message back to the agent.
5. Ask the agent to repair only the code.
6. Execute again.

Limit repair attempts to two or three cycles.

Track failure types:

wrong column name

missing import

syntax error

unsupported chart type

empty output file

timeout

memory error

This matches the style of the related course material, where failures are classified rather than treated as generic errors.

Phase 6: Add evaluation benchmark

Create a test set similar to the tool-routing benchmark idea in your course files, where tasks are stored as structured JSONL records and scored automatically.

Example:

```
{  
  "id": "viz-001",  
  "prompt": "Create a bar chart of total sales by region",  
  "dataset": "sales.csv",  
  "expectedChartType": "bar",  
  "expectedColumns": ["Region", "Sales"],  
  "expectedOutputCount": 1  
}
```

Measure:

code execution success rate

correct chart type rate

correct column mapping rate

average execution time

multi-chart success rate

number of repair attempts

large dataset execution time

7. Suggested project milestones

Milestone Goal		Output
M1	Requirements and architecture	diagrams, use cases, scope
M2	Basic Microsoft Agent Framework app	agent responds and generates code
M3	Custom Code Interpreter Tool	Python code executes from C#
M4	Dataset profiler	schema and sample extraction
M5	Visualization planner	structured chart plan JSON
M6	Code validation and repair	safer and more reliable execution
M7	Multi-chart and large dataset support	complex user prompts handled
M8	Benchmark and report	metrics, screenshots, final demo

8. Main risks and solutions

Risk	Solution
LLM hallucinates column names	Use dataset profiler and validate columns before execution
Generated code is unsafe	Run in sandbox, restrict imports, disable network, use timeout
Large dataset cannot fit in prompt	Send schema/statistics only; pass file path to Python
Python code fails	Add repair loop using stderr
Wrong chart type	Use structured visualization plan before code generation
Multiple charts become messy	Generate one chart spec per output file
Model chooses wrong tool	Use clear tool descriptions and explicit tool-call logging

9. Recommended final demo

A good final demo could show three scenarios:

Demo 1:

Upload small CSV.

Prompt: “Create a bar chart of total sales by region.”

Output: one PNG chart.

Demo 2:

Upload dataset with date column.

Prompt: “Show monthly revenue trend and profit by region.”

Output: two charts.

Demo 3:

Upload large CSV.

Prompt: “Create a summary visualization of average sales by category.”

Output: aggregated chart without sending all rows to the LLM.

Also show:

generated Python code

execution logs

output files

error handling example

benchmark metrics

10. Final recommendation

Build the project as a **controlled LLM-to-Python visualization pipeline**, not as an unrestricted code generator. The strongest version will have:

Microsoft Agent Framework agent in .NET/C#

custom Code Interpreter Tool

Python sandbox execution

dataset profiling

structured visualization planning

code validation

automatic repair loop

benchmark-based evaluation

That will directly address your professor's key themes: agent architecture, robust user-data mapping, dynamic Python execution, scalability, multiple visualization generation, and reliable handling of model-generated code.